

Pq

堆

__gnu_pbds::priority_queue

附：[官方文档地址——复杂度及常数测试](#)

```
1 #include <libstdc++/pb_ds/priority_queue.hpp>
2 using namespace __gnu_pbds;
3 __gnu_pbds::priority_queue<T, Compare, Tag, Allocator>
```

模板形参

- T: 储存的元素类型
- Compare: 提供严格的弱序比较类型
- Tag: 是 __gnu_pbds 提供的不同的五种堆，Tag 参数默认是 pairing_heap_tag 五种分别是：
 - pairing_heap_tag: 配对堆 官方文档认为在非原生元素（如自定义结构体/std::string/pair）中，配对堆表现最好
 - binary_heap_tag: 二叉堆 官方文档认为在原生元素中二叉堆表现最好，不过笔者测试的表现并没有那么好
 - binomial_heap_tag: 二项堆 二项堆在合并操作的表现要优于二叉堆，但是其取堆顶元素操作的复杂度比二叉堆高
 - rc_binomial_heap_tag: 冗余计数二项堆
 - thin_heap_tag: 除了合并的复杂度都和 Fibonacci 堆一样的一个 tag
- Allocator: 空间配置器，由于 OI 中很少出现，故这里不做讲解

由于本篇文章只是提供给学习算法竞赛的同学们，故对于后四个 tag 只会简单的介绍复杂度，第一个会介绍成员函数和使用方法。

经作者本机 Core i5 @3.1 GHz On macOS 测试堆的基础操作，结合 GNU 官方的复杂度测试，Dijkstra 测试，都表明：至少对于 Oier 来讲，除了配对堆的其他四个 tag 都是鸡肋，要么没用，要么常数大到不如 std 的，且有可能造成 MLE，故这里只推荐用默认的配对堆。同样，配对堆也优于 algorithm 库中的 make_heap()。

构造方式

要注明命名空间因为和 std 的类名称重复。

```
1 // __gnu_pbds::priority_queue<LTint>;
2 // __gnu_pbds::priority_queue<LTint, greater<LTint>>;
3 // __gnu_pbds::priority_queue<LTint, greater<LTint>, pairing_heap_tag>;
4 __gnu_pbds::priority_queue<int>::point_iterator id; // 点类型迭代器
5 // 在 modify 和 push 的时候都会返回一个 point_iterator，下文会详细的讲使用方法
6 id = q.push(1);
```

成员函数

- push(): 向堆中压入一个元素，返回该元素位置的迭代器。
- pop(): 将堆顶元素弹出。
- top(): 返回堆顶元素。

- `size()` 返回元素个数。
- `empty()` 返回是否非空。
- `modify(point_iterator, const key)`: 把迭代器位置的 `key` 修改为传入的 `key`, 并对底层储存结构进行排序。
- `erase(point_iterator)`: 把迭代器位置的键值从堆中擦除。
- `join(__gnu_pbds::priority_queue &other)`: 把 `other` 合并到 `*this` 并把 `other` 清空。

使用的 `tag` 决定了每个操作的时间复杂度:

	push	pop	modify	erase	Join
<code>pairing_heap_tag</code>		最坏 均摊	最坏 均摊	最坏 均摊	
<code>binary_heap_tag</code>	最坏 均摊	最坏 均摊			
<code>binomial_heap_tag</code>	最坏 均摊				
<code>rc_binomial_heap_tag</code>					
<code>thin_heap_tag</code>		最坏 均摊	最坏 均摊	最坏 均摊	

示例

```

1 #include <LTalgorithm>
2 #include <LTcstdio>
3 #include <LText/pb_ds/priority_queue.hpp>
4 #include <LTiostream>
5 using namespace __gnu_pbds;
6 // 由于面向Oier, 本文以常用堆 : pairing_heap_tag作为范例
7 // 为了更好的阅读体验, 定义宏如下 :
8 using pair_heap = __gnu_pbds::priority_queue<int>;
9 pair_heap q1; // 大根堆, 配对堆
10 pair_heap q2;
11 pair_heap::point_iterator id; // 一个迭代器
12
13 int main() {
14     id = q1.push(1);
15     // 堆中元素 : [1];
16     for (int i = 2; i <= 5; i++) q1.push(i);
17     // 堆中元素 : [1, 2, 3, 4, 5];
18     std::cout << q1.top() << std::endl;
19     // 输出结果 : 5;
20     q1.pop();
21     // 堆中元素 : [1, 2, 3, 4];
22     id = q1.push(10);
23     // 堆中元素 : [1, 2, 3, 4, 10];
24     q1.modify(id, 1);
25     // 堆中元素 : [1, 1, 2, 3, 4];
26     std::cout << q1.top() << std::endl;
27     // 输出结果 : 4;
28     q1.pop();
29     // 堆中元素 : [1, 1, 2, 3];
30     id = q1.push(7);
31     // 堆中元素 : [1, 1, 2, 3, 7];
32     q1.erase(id);
33     // 堆中元素 : [1, 1, 2, 3];
34     q2.push(1), q2.push(3), q2.push(5);
35     // q1中元素 : [1, 1, 2, 3], q2中元素 : [1, 3, 5];
36     q2.join(q1);
37     // q1中无元素, q2中元素 : [1, 1, 1, 2, 3, 3, 5];
38 }

```

__gnu_pbds 迭代器的失效保证 (invalidation_guarantee)

在上述示例以及一些实践中（如使用本章的 pb-ds 堆来编写单源最短路等算法），常常需要保存并使用堆的迭代器（如 `__gnu_pbds::priority_queue<LTint>::point_iterator` 等）。

可是例如对于 `__gnu_pbds::priority_queue` 中不同的 Tag 参数，其底层实现并不相同，迭代器的失效条件也不一样，根据 `__gnu_pbds` 库的设计，以下三种由上至下派生的情况：

1. 基本失效保证 (`basic_invalidation_guarantee`)：即不修改容器时，点类型迭代器 (`point_iterator`)、指针和引用 (`key/value`) 保持有效。
2. 点失效保证 (`point_invalidation_guarantee`)：即修改容器后，点类型迭代器 (`point_iterator`)、指针和引用 (`key/value`) 只要对应容器中没被删除保持有效。
3. 范围失效保证 (`range_invalidation_guarantee`)：即修改容器后，除 (2) 的特性以外，任何范围类型的迭代器（包括 `begin()` 和 `end()` 的返回值）是正确的，具有范围失效保证的 Tag 有 `rb_tree_tag` 和 适用于 `__gnu_pbds::tree` 的 `splay_tree_tag`，以及 适用于 `__gnu_pbds::trie` 的 `pat_trie_tag`。

从运行下述代码中看出，除了 `binary_heap_tag` 为 `basic_invalidation_guarantee` 在修改后迭代器会失效，其余的均为 `point_invalidation_guarantee` 可以实现修改后点类型迭代器 (`point_iterator`) 不失效的需求。

```
1 #include <LTIostream>
2 using namespace std;
3 #include <LText/pb_ds/assoc_container.hpp>
4 #include <LText/pb_ds/priority_queue.hpp>
5 using namespace __gnu_pbds;
6 #include <LTcxxabi.h>
7
8 template <typename T>
9 void print_invalidation_guarantee() {
10     using gute = __gnu_pbds::container_traits<T>::invalidation_guarantee;
11     cout << abi::__cxa_demangle(typeid(gute).name(), 0, 0, 0) << endl;
12 }
13
14 int main() {
15     using pairing =
16         __gnu_pbds::priority_queue<int, greater<int>, pairing_heap_tag>;
17     using binary = __gnu_pbds::priority_queue<int, greater<int>, binary_heap_tag>;
18     using binomial =
19         __gnu_pbds::priority_queue<int, greater<int>, binomial_heap_tag>;
20     using rc_binomial =
21         __gnu_pbds::priority_queue<int, greater<int>, rc_binomial_heap_tag>;
22     using thin = __gnu_pbds::priority_queue<int, greater<int>, thin_heap_tag>;
23     print_invalidation_guarantee<pairing>();
24     print_invalidation_guarantee<binary>();
25     print_invalidation_guarantee<binomial>();
26     print_invalidation_guarantee<rc_binomial>();
27     print_invalidation_guarantee<thin>();
28     return 0;
29 }
```

本页面最近更新：2024/10/9 22:38:42, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [GoodCoder666](#), [lr1d](#), [opsiff](#), [ouuan](#), [Xeonacid](#), [abc1763613206](#), [Chrogeek](#), [CoderOJ](#), [countercurrent-time](#), [Early0v0](#), [Enter-tainer](#), [H-J-Granger](#), [i-Yirannn](#), [ksyx](#), [NachtgeistW](#), [Planet6174](#), [SukkaW](#), [Tiphereth-A](#), [WAAutoMaton](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用

